PYTHON ACTIVITY-BASED LEARNING

A Collection of 10 Python Case Studies

Student Name	Thota Hima Sri
Roll Number	26001252
College	NRI UNIVERSITY
Branch	CSM
Year	1st Year
Subject	Python Programming

Submitted as part of the Python Programming course.

Table of Contents

1. Progressive Electricity Billing
2. Employee Payroll and Deduction Analysis
3. Retail Billing with Multiple Discount Rules
4. Attendance Recovery Computation
5. ATM Transaction Sequence Simulation
6. Password Policy Validation Engine
7. Text Compression Using Character Runs
8. Nested Loop Calendar Generator
9. Student Marks Matrix Analysis
10. Tuple-Based Employee Shift Conflict Detection

Case Study 1: Progressive Electricity Billing

1. Problem Statement

Calculate an electricity bill using progressive unit rates.

2. Objectives

• Apply different rates to unit slabs.

• Practise variables, arithmetic and conditions.

3. Assumptions

• First 100 units: ■2 each; next 100: ■3 each; above 200: ■5 each.

• No fixed charge or tax; negative units are invalid.

4. Algorithm

• Read units.

• Reject negative units.

• For up to 100, charge units × 2.

• For up to 200, charge 200 + (units – 100) × 3.

• Above 200, charge 500 + (units – 200) × 5.

• Display bill.

5. Flowchart

START → Input units → Negative? → Invalid : Check slab → Calculate bill → Display → STOP

6. Pseudocode

```
INPUT units; IF units<0: invalid; ELIF units<=100: bill=units*2; ELIF units<=200: bill=200+(units-100)*3; ELSE: bill=500+(units-200)*5;
```

7. Python Source Code

```
units = int(input("Enter units: "))
if units < 0:
    print("Invalid units")
elif units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 200 + (units - 100) * 3
else:
    bill = 500 + (units - 200) * 5
if units >= 0:
    print("Electricity bill: Rs.", bill)
```

8. Test Cases

Test Input	Expected Result
50	■100
100	■200
150	■350
250	■750
-5	Invalid units

9. Sample Program Output

```
Enter units: 250
Electricity bill: Rs. 750
```

10. Conclusion

Progressive billing is calculated using arithmetic and if-elif-else statements.

Case Study 2: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate an employee's gross and net salary from basic salary, allowances and deductions.

2. Objectives

• Calculate percentage-based salary components.

• Practise arithmetic and conditions.

3. Assumptions

• HRA = 20% of basic; DA = 10% of basic; deduction = 5% of gross.

• Salary cannot be negative.

4. Algorithm

• Read employee name and basic salary.

• Reject negative salary.

• Calculate HRA and DA.

• Calculate gross salary.

• Calculate 5% deduction and net salary.

• Display payslip.

5. Flowchart

START → Input details → Validate salary → Calculate HRA/DA → Calculate gross and deduction → Calculate net → Display → STOP

6. Pseudocode

```
INPUT name,basic; IF basic<0: invalid; ELSE hra=basic*0.20; da=basic*0.10; gross=basic+hra+da; deduction=gross*0.05; net=gross-deductio
```

7. Python Source Code

```
name = input("Employee name: ")
basic = float(input("Basic salary: "))
if basic < 0:
    print("Invalid salary")
else:
    hra = basic * 0.20
    da = basic * 0.10
    gross = basic + hra + da
    deduction = gross * 0.05
    net = gross - deduction
    print("Employee:", name)
    print("Gross salary: Rs.", round(gross, 2))
    print("Deduction: Rs.", round(deduction, 2))
    print("Net salary: Rs.", round(net, 2))
```

8. Test Cases

Test Input	Expected Result
■20,000	Gross ■26,000; deduction ■1,300; net ■24,700
■30,000	Gross ■39,000; deduction ■1,950; net ■37,050
-500	Invalid salary

9. Sample Program Output

```
Employee name: Ravi
Basic salary: 20000
Gross salary: Rs. 26000.0
Deduction: Rs. 1300.0
Net salary: Rs. 24700.0
```

10. Conclusion

The payroll program calculates allowances, deductions and net salary using arithmetic and conditions.

Case Study 3: Retail Billing with Multiple Discount Rules

1. Problem Statement

Read item prices, calculate the total and apply a discount based on the purchase amount.

2. Objectives

• Store prices in a list.

• Use a loop to total prices and conditions to select a discount.

3. Assumptions

• Total ≥ ■5000: 10% discount; ■2000–■4999.99: 5%; below ■2000: no discount.

• Prices must not be negative.

4. Algorithm

• Read item count and create an empty list.

• Input each price and append it.

• Calculate subtotal using sum().

• Choose discount rate using conditions.

• Calculate and display final amount.

5. Flowchart

START → Input count → Loop: collect prices → Sum prices → Check discount slab → Calculate final amount → Display → STOP

6. Pseudocode

```
INPUT count; prices=[]; REPEAT count times: INPUT price; APPEND price; subtotal=SUM(prices); IF subtotal>=5000 rate=.10; ELIF subtotal>
```

7. Python Source Code

```
count = int(input("Number of items: "))
prices = []
for i in range(count):
    price = float(input("Price of item " + str(i + 1) + ": "))
    if price < 0:
        print("Invalid price")
        break
    prices.append(price)
else:
    subtotal = sum(prices)
    if subtotal >= 5000:
        rate = 0.10
    elif subtotal >= 2000:
        rate = 0.05
    else:
        rate = 0
    discount = subtotal * rate
    print("Subtotal: Rs.", subtotal)
    print("Discount: Rs.", discount)
    print("Final amount: Rs.", subtotal - discount)
```

8. Test Cases

Test Input	Expected Result
■1000 + ■1200	Subtotal ■2200; discount ■110; final ■2090
■3000 + ■2500	Subtotal ■5500; discount ■550; final ■4950
■500 + ■700	Subtotal ■1200; no discount

9. Sample Program Output

```
Number of items: 2
Price of item 1: 3000
Price of item 2: 2500
Subtotal: Rs. 5500.0
Discount: Rs. 550.0
Final amount: Rs. 4950.0
```

10. Conclusion

The billing program uses a list, loop and conditions to calculate discounts.

Case Study 4: Attendance Recovery Computation

1. Problem Statement

Calculate attendance percentage and find the additional consecutive classes needed to reach 75%.

2. Objectives

• Calculate attendance percentage.

• Use a while loop to find required additional classes.

3. Assumptions

• Required attendance is 75%.

• Conducted classes must be positive and attended cannot exceed conducted.

4. Algorithm

• Input conducted and attended classes.

• Validate values.

• Calculate percentage.

• While percentage is below 75, add one attended and one conducted class.

• Display current percentage and extra classes.

5. Flowchart

START → Input classes → Validate → Calculate percentage → Below 75? → Loop adding attended class and conducted class → Display extra classes → STOP

6. Pseudocode

```
INPUT conducted,attended; validate; percentage=attended/conducted*100; extra=0; WHILE percentage<75: conducted+=1; attended+=1; extra+=
```

7. Python Source Code

```
conducted = int(input("Total classes conducted: "))
attended = int(input("Classes attended: "))
if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance details")
else:
    original = attended / conducted * 100
    extra = 0
    while attended / conducted * 100 < 75:
        attended += 1
        conducted += 1
        extra += 1
    print("Current attendance:", round(original, 2), "%")
    print("Additional classes required:", extra)
    print("Attendance after recovery:", round(attended / conducted * 100, 2), "%")
```

8. Test Cases

Test Input	Expected Result
100 conducted, 60 attended	60%; 60 additional classes
80 conducted, 64 attended	80%; 0 additional classes
50 conducted, 30 attended	60%; 30 additional classes

9. Sample Program Output

```
Total classes conducted: 100
Classes attended: 60
Current attendance: 60.0 %
Additional classes required: 60
Attendance after recovery: 75.0 %
```

10. Conclusion

The attendance program uses a while loop and conditions to calculate recovery classes.

Case Study 5: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate deposits, withdrawals, balance enquiry and exit; record valid transactions in a list.

2. Objectives

• Use a list to store transaction history.

• Practise loops, conditions and balance updates.

3. Assumptions

• Starting balance is ■5000.

• Amounts must be positive; withdrawal cannot exceed balance.

4. Algorithm

• Set balance to 5000 and transactions to an empty list.

• Display menu repeatedly.

• Deposit: add amount and record it.

• Withdraw: validate amount and balance, then subtract and record.

• Balance: display balance; Exit: stop.

• Display history and final balance.

5. Flowchart

START → Set balance/list → Menu → Deposit / Withdraw / Balance / Exit → Update and record → Repeat until Exit → Display history → STOP

6. Pseudocode

```
balance=5000; transactions=[]; WHILE true: INPUT choice; IF deposit validate/add; ELIF withdraw validate/subtract; ELIF balance print;
```

7. Python Source Code

```
balance = 5000.0
transactions = []
while True:
    print("\n1.Deposit 2.Withdraw 3.Balance 4.Exit")
    choice = input("Choice: ")
    if choice == "1":
        amount = float(input("Deposit amount: "))
        if amount > 0:
            balance += amount
            transactions.append("Deposit: " + str(amount))
        else:
            print("Enter a positive amount")
    elif choice == "2":
        amount = float(input("Withdrawal amount: "))
        if amount <= 0:
            print("Enter a positive amount")
        elif amount > balance:
            print("Insufficient balance")
        else:
            balance -= amount
            transactions.append("Withdrawal: " + str(amount))
    elif choice == "3":
        print("Balance: Rs.", balance)
    elif choice == "4":
        break
    else:
        print("Invalid choice")
print("Transactions:", transactions)
print("Final balance: Rs.", balance)
```

8. Test Cases

Test Input	Expected Result
Deposit ■1000	Balance becomes ■6000
Withdraw ■1500	Balance becomes ■3500
Withdraw ■7000	Insufficient balance
Choose 4	Session ends

9. Sample Program Output

```
Choice: 1
Deposit amount: 1000
Choice: 3
Balance: Rs. 6000.0
Choice: 4
Transactions: ['Deposit: 1000.0']
Final balance: Rs. 6000.0
```

10. Conclusion

The ATM simulation uses a menu loop, conditions and a list to maintain transaction history.

Case Study 6: Password Policy Validation Engine

1. Problem Statement

Check whether a password contains at least eight characters, an uppercase letter, a lowercase letter and a digit.

2. Objectives

• Practise string operations and character checks.

• Use a loop and Boolean variables.

3. Assumptions

• Minimum length is 8.

• At least one uppercase, lowercase and digit are required.

• Special characters are optional.

4. Algorithm

• Input password.

• Set uppercase, lowercase and digit flags to False.

• Check every character and update flags.

• Check length and all flags.

• Display valid or invalid.

5. Flowchart

START → Input password → Set flags False → Loop through characters → Update flags → Check length and flags → Display result → STOP

6. Pseudocode

```
INPUT password; upper=lower=digit=false; FOR each character: check isupper/islower/isdigit; IF length>=8 AND all flags: valid ELSE inva
```

7. Python Source Code

```
password = input("Enter password: ")
has_upper = has_lower = has_digit = False
for ch in password:
    if ch.isupper():
        has_upper = True
    elif ch.islower():
        has_lower = True
    elif ch.isdigit():
        has_digit = True
if len(password) >= 8 and has_upper and has_lower and has_digit:
    print("Valid password")
else:
    print("Invalid password")
    print("Use 8+ characters with uppercase, lowercase and a digit.")
```

8. Test Cases

Test Input	Expected Result
Hello123	Valid password
hello123	Invalid: no uppercase
HELLO123	Invalid: no lowercase
HelloWorld	Invalid: no digit
Hi1	Invalid: too short

9. Sample Program Output

```
Enter password: Hello123
Valid password
```

10. Conclusion

The validator uses strings, loops and conditions to check password requirements.

Case Study 7: Text Compression Using Character Runs

1. Problem Statement

Compress consecutive repeated characters by writing each character followed by its repetition count, such as AAABB becoming A3B2.

2. Objectives

• Practise string indexing and traversal.

• Build a result using concatenation and a loop.

3. Assumptions

• Input is case-sensitive and contains no spaces.

• Every run includes its count, even when the count is 1.

4. Algorithm

• Input text and create an empty result.

• If not empty, set count to 1.

• Compare each character with the previous one.

• Increase count for a match; otherwise append previous character and count.

• Append the final run and display.

5. Flowchart

START → Input text → Empty? → Return empty → Otherwise compare adjacent characters in loop → Count repeats or append run → Append final run → Display → STOP

6. Pseudocode

```
INPUT text; result=''; IF text not empty: count=1; FOR i from 1: IF text[i]==text[i-1] count++; ELSE append previous+count and reset; a
```

7. Python Source Code

```
text = input("Enter text (no spaces): ")
compressed = ""
if text:
    count = 1
    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            compressed += text[i - 1] + str(count)
            count = 1
    compressed += text[-1] + str(count)
print("Compressed text:", compressed)
```

8. Test Cases

Test Input	Expected Result
AAABB	A3B2
WWWWAA	W4A2
ABC	A1B1C1
A	A1
Empty input	Empty result

9. Sample Program Output

```
Enter text (no spaces): AAABB
Compressed text: A3B2
```

10. Conclusion

The program demonstrates string processing, loops, indexing and concatenation.

Case Study 8: Nested Loop Calendar Generator

1. Problem Statement

Display a simple month calendar from the number of days and the weekday on which the month starts.

2. Objectives

• Practise loops and pattern generation.

• Use conditions to print calendar rows.

3. Assumptions

• Days must be 28–31.

• Start weekday is 0=Sunday through 6=Saturday.

• No month/year calculation is performed.

4. Algorithm

• Input days and starting weekday.

• Validate both values.

• Print weekday headings and leading blank cells.

• Loop through dates and print each in a fixed-width column.

• Start a new row after Saturday.

5. Flowchart

START → Input days/start weekday → Validate → Print headings and blanks → Loop dates → Print date → End of week? New row → STOP

6. Pseudocode

```
INPUT days,start; validate; PRINT headings; print start blank cells; FOR date=1..days: print date; IF (start+date) MOD 7=0: newline; ST
```

7. Python Source Code

```
days = int(input("Days in month (28 to 31): "))
start = int(input("Starting weekday (0=Sun, 6=Sat): "))
if days < 28 or days > 31 or start < 0 or start > 6:
    print("Invalid calendar details")
else:
    print("Sun Mon Tue Wed Thu Fri Sat")
    for blank in range(start):
        print("    ", end="")
    for date in range(1, days + 1):
        print(f"{date:3}", end=" ")
        if (start + date) % 7 == 0:
            print()
    print()
```

8. Test Cases

Test Input	Expected Result
30 days, start 0	Dates begin under Sunday
31 days, start 3	Dates begin under Wednesday
27 days	Invalid calendar details
30 days, start 7	Invalid calendar details

9. Sample Program Output

```
Days in month (28 to 31): 30
Starting weekday (0=Sun, 6=Sat): 0
Sun Mon Tue Wed Thu Fri Sat
  1   2   3   4   5   6   7
  8   9  10  11  12  13  14
 15  16  17  18  19  20  21
 22  23  24  25  26  27  28
 29  30
```

10. Conclusion

The calendar program generates a table-like pattern using loops, conditions and formatted output.

Case Study 9: Student Marks Matrix Analysis

1. Problem Statement

Store marks of students in a nested list and calculate student totals, averages and subject averages.

2. Objectives

• Represent tabular data using nested lists.

• Use nested loops to process rows and columns.

3. Assumptions

• Three students and three subjects are used.

• Marks are between 0 and 100.

• Rows represent students; columns represent subjects.

4. Algorithm

• Create a nested list of marks.

• For each student row, calculate total and average.

• For each subject column, add marks across students.

• Calculate and display subject averages.

5. Flowchart

START → Create marks matrix → Loop student rows → Calculate totals/averages → Loop subject columns → Calculate subject averages → Display → STOP

6. Pseudocode

```
marks=[[80,75,90],[65,85,70],[90,95,88]]; FOR each row: total=SUM(row), average=total/3; FOR each column: sum values across rows and di
```

7. Python Source Code

```
marks = [[80, 75, 90], [65, 85, 70], [90, 95, 88]]
names = ["Anu", "Balu", "Chandu"]
subjects = ["Python", "Maths", "English"]

for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])
    print(names[i], "Total:", total, "Average:", round(average, 2))

print("Subject averages:")
for col in range(len(subjects)):
    total = 0
    for row in range(len(marks)):
        total += marks[row][col]
    print(subjects[col], ":", round(total / len(marks), 2))
```

8. Test Cases

Test Input	Expected Result
Anu: 80,75,90	Total 245; average 81.67
Balu: 65,85,70	Total 220; average 73.33
Chandu: 90,95,88	Total 273; average 91.00
Python marks 80,65,90	Subject average 78.33

9. Sample Program Output

```
Anu Total: 245 Average: 81.67
Balu Total: 220 Average: 73.33
Chandu Total: 273 Average: 91.0
Subject averages:
Python : 78.33
Maths : 85.0
English : 82.67
```

10. Conclusion

The nested-list program processes student rows and subject columns to calculate useful statistics.

Case Study 10: Tuple-Based Employee Shift Conflict Detection

1. Problem Statement

Store employee shifts as tuples and determine whether two shifts overlap.

2. Objectives

• Create and access tuples.

• Use tuple unpacking.

• Compare time ranges using conditions.

3. Assumptions

• Shift format is (employee_name, start_hour, end_hour).

• Hours are whole numbers from 0 to 23; overnight shifts are excluded.

• Two shifts overlap when start1 < end2 and start2 < end1.

4. Algorithm

• Create two shift tuples.

• Unpack each tuple into name, start and end.

• Validate time ranges.

• Apply the overlap condition.

• Display conflict or no conflict.

5. Flowchart

START → Create tuples → Unpack values → Validate hours → Check start1<end2 AND start2<end1 → Conflict or no conflict → STOP

6. Pseudocode

```
shift1=(name1,start1,end1); shift2=(name2,start2,end2); UNPACK; validate; IF start1<end2 AND start2<end1: conflict ELSE no conflict.
```

7. Python Source Code

```
shift1 = ("Anu", 9, 13)
shift2 = ("Balu", 12, 16)
name1, start1, end1 = shift1
name2, start2, end2 = shift2

if not (0 <= start1 < end1 <= 23 and 0 <= start2 < end2 <= 23):
    print("Invalid shift timing")
elif start1 < end2 and start2 < end1:
    print("Shift conflict detected")
    print(name1, "and", name2, "have overlapping shifts.")
else:
    print("No shift conflict")
```

8. Test Cases

Test Input	Expected Result
Anu 9–13; Balu 12–16	Conflict detected
Anu 9–13; Balu 13–17	No conflict
Anu 8–12; Balu 14–18	No conflict
Start 15, end 10	Invalid shift timing

9. Sample Program Output

```
Shift conflict detected
Anu and Balu have overlapping shifts.
```

10. Conclusion

The program uses tuples and tuple unpacking to compare employee shift times.